COMPUTER ARCHITECTURE

Laboratory Report

RISC-V Single-Cycle Processor on FPGA

Tasks A & B: Countdown Timer and Control Signal Verification

April 2026

Platform: Digilent Basys 3 (Xilinx Artix-7)

HDL: Verilog (Vivado 2024)

GIT LINK: https://github.com/sohaibm05/LABS.git

Sohaib Muhammad (09751) • Muhammad Abdullah Khan (10022) • Syed Ibad Asim (09753)

1. Introduction

This report documents the implementation and hardware verification of a 32-bit RISC-V single-cycle processor synthesised onto a Digilent Basys 3 FPGA development board. Two tasks are covered: Task A demonstrates the processor executing a countdown timer subroutine, with the Program Counter (PC) value displayed in real time on the on-board 7-segment display and the countdown value shown on the 16 LEDs. Task B verifies correct generation of the processor's control signals by loading a purpose-built instruction sequence that exercises the LUI, JAL, and JALR instructions, with the resulting control flags observable on the LEDs in debug mode.

The processor implements the RV32I base integer instruction set, supporting R-type arithmetic, I-type (including loads and immediate arithmetic), S-type stores, B-type conditional branches, J-type unconditional jumps, and U-type instructions. Memory-mapped I/O is used to interface with the board's switches (read address 0x300) and LEDs (write address 0x200).

2. System Architecture

The top-level module, TopLevelProcessor, instantiates and connects all datapath and control components in a strictly single-cycle fashion: every instruction completes in one clock cycle. A 26-bit clock divider reduces the 100 MHz system clock down to approximately 1 Hz, giving each instruction a visible, human-scale execution window. The 7-segment display controller runs on the full 100 MHz clock so that the multiplexed display remains stable.

2.1 Datapath Overview

The five canonical stages of the datapath are all combinationally active in the same cycle:

- Instruction Fetch — ProgramCounter, pcAdder, instructionMemory
- Decode & Control — MainControl, ALUControl, immGen, RegisterFile
- Execute — ALU, branchAdder, branch condition logic, PC mux
- Memory & I/O — DataMemory, AddressDecoder, leds module
- Write Back — MemtoReg mux writing to RegisterFile on the next rising edge

2.2 Clock Strategy

Two clock domains coexist in the design. The slow_clk (≈1 Hz) drives the PC, register file, and data memory, allowing each instruction to be observed in isolation on the hardware. The fast clk (100 MHz) drives only the SevenSegmentControl module, ensuring flicker-free display multiplexing. This split avoids the timing complexity of a single slow domain while still providing a human-observable execution rate.

```
// Clock divider: 100 MHz -> ~1 Hz
reg [25:0] clk_counter = 0;
reg slow_clk = 0;
always @(posedge clk) begin
    if (clk_counter == 49999999) begin
        clk_counter <= 0;
        slow_clk <= ~slow_clk;
    end else
        clk_counter <= clk_counter + 1;
end
```

2.3 I/O Memory Map

The AddressDecoder module uses bits [9:8] of the ALU result to route memory transactions to the correct peripheral:

Address Range	Bits [9:8]	Peripheral
`0x000–0x1FF`	`00`	Data Memory (512 words)
`0x200–0x2FF`	`10`	LED Output Register
`0x300–0x3FF`	`11`	Switch Input (read-only)

3. Task A — Countdown Timer with 7-Segment Display

3.1 Objective

Task A required implementing a RISC-V assembly program that reads a value from the hardware switches, displays it on the LEDs, calls a countdown subroutine that decrements the value to zero (with each step visible on the LEDs), and then idles until the switches are cleared. The current Program Counter is simultaneously shown on the 7-segment display, providing direct visibility into instruction flow during execution.

3.2 Program Flow

The assembly program is structured into four logical sections:

- Setup (0x00–0x0B): Initialises the stack pointer, switch address register (x5 = 0x300), and LED address register (x6 = 0x200), then clears the LEDs on startup.
- IDLE Loop (0x10–0x28): Polls the switches and waits for a non-zero value, then writes the switch value to LEDs and calls COUNTDOWN_SUB via JAL.
- WAIT_RELEASE (0x3C–0x44): After the countdown completes, spins until the switches return to zero before jumping back to IDLE.
- COUNTDOWN_SUB Subroutine (0x50–0x98): Saves callee-saved registers on the stack, decrements a counter register each iteration (with an inner software delay loop), writes the current count to LEDs, and returns via JALR.

3.3 Key Code — Instruction Memory Initialisation

The RISC-V machine code is loaded directly into the instruction memory's byte array. Below is the complete initialisation block for the Countdown Timer program:

```
// ============= TASK A — COUNTDOWN TIMER =============
initial begin
    for (i = 0; i < 256; i = i + 1) memory[i] = 8'h00;

    // --- SETUP ---
    // 0x00: addi x2, x0, 252     # sp = 0xFC
    memory[0]=8'h13; memory[1]=8'h01; memory[2]=8'hC0; memory[3]=8'h0F;
    // 0x04: addi x5, x0, 768     # x5 = switch address (0x300)
    memory[4]=8'h93; memory[5]=8'h02; memory[6]=8'h00; memory[7]=8'h30;
    // 0x08: addi x6, x0, 512     # x6 = LED address (0x200)
    memory[8]=8'h13; memory[9]=8'h03; memory[10]=8'h00; memory[11]=8'h20;
    // 0x0C: sw x0, 0(x6)         # clear LEDs on startup
    memory[12]=8'h23; memory[13]=8'h20; memory[14]=8'h03; memory[15]=8'h00;

    // --- IDLE LOOP ---
    // 0x10: lw x7, 0(x5)         # read switches
    memory[16]=8'h83; memory[17]=8'hA3; memory[18]=8'h02; memory[19]=8'h00;
    // 0x14: beq x7, x0, -4       # if 0, loop back to IDLE
    memory[20]=8'hE3; memory[21]=8'h8E; memory[22]=8'h03; memory[23]=8'hFE;
    // 0x18: sw x7, 0(x6)         # display switch value on LEDs
    memory[24]=8'h23; memory[25]=8'h20; memory[26]=8'h73; memory[27]=8'h00;
    // 0x1C: add x10, x7, x0      # x10 = argument
    memory[28]=8'h33; memory[29]=8'h85; memory[30]=8'h03; memory[31]=8'h00;
    // 0x2C: jal x1, +36          # call COUNTDOWN_SUB at 0x50
    memory[44]=8'hEF; memory[45]=8'h00; memory[46]=8'h40; memory[47]=8'h02;

    // --- COUNTDOWN_SUB SUBROUTINE (0x50) ---
```

```
    // Saves x1, x5, x6, x10 on stack; decrements x10 each iteration
    // 0x64: sw x10, 0(x6)        # leds = current count
    memory[100]=8'h23; memory[101]=8'h20; memory[102]=8'hA3; memory[103]=8'h00;
    // 0x6C: addi x8, x0, 500     # delay counter
    memory[108]=8'h13; memory[109]=8'h04; memory[110]=8'h40; memory[111]=8'h1F;
    // 0x78: addi x10, x10, -1    # count--
    memory[120]=8'h13; memory[121]=8'h05; memory[122]=8'hF5; memory[123]=8'hFF;
    // 0x98: jalr x0, x1, 0       # return
    memory[152]=8'h67; memory[153]=8'h80; memory[154]=8'h00; memory[155]=8'h00;
end
```

3.4 7-Segment Display Controller

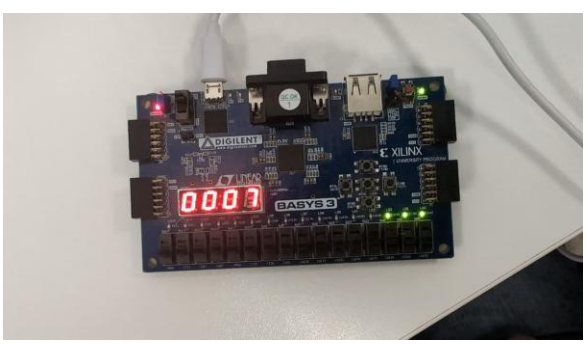
The SevenSegmentControl module multiplexes a 16-bit value across the four display digits using a 20-bit free-running counter driven by the 100 MHz clock. The upper two bits of this counter (bits [19:18]) cycle through the four digit positions at approximately 380 Hz, well above the persistence-of-vision threshold. When sw[15] is high (debug mode), the lower 16 bits of the PC are shown; otherwise the LED output register is displayed.

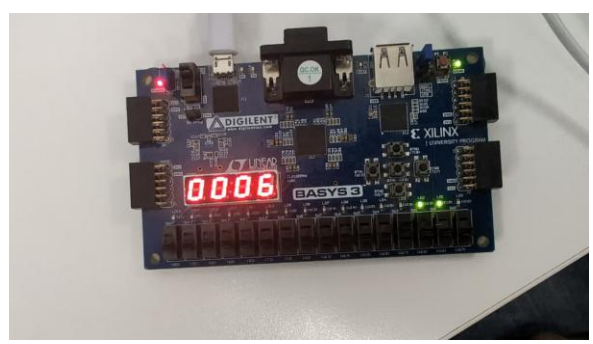
```
// In TopLevelProcessor:
wire [15:0] display_value = sw[15] ? PC[15:0] : leds;

SevenSegmentControl display_unit (
    .clk(clk),   // Must be the fast 100 MHz clock
    .rst(rst),
    .value(display_value),
    .an(an),
    .seg(seg)
);
```

3.5 Hardware Results

The following photographs were taken during live execution on the Basys 3. With sw[15] set high (debug mode), the 7-segment display shows the PC value advancing by 4 with each instruction. The LEDs reflect the current countdown value written to the LED register at address 0x200.





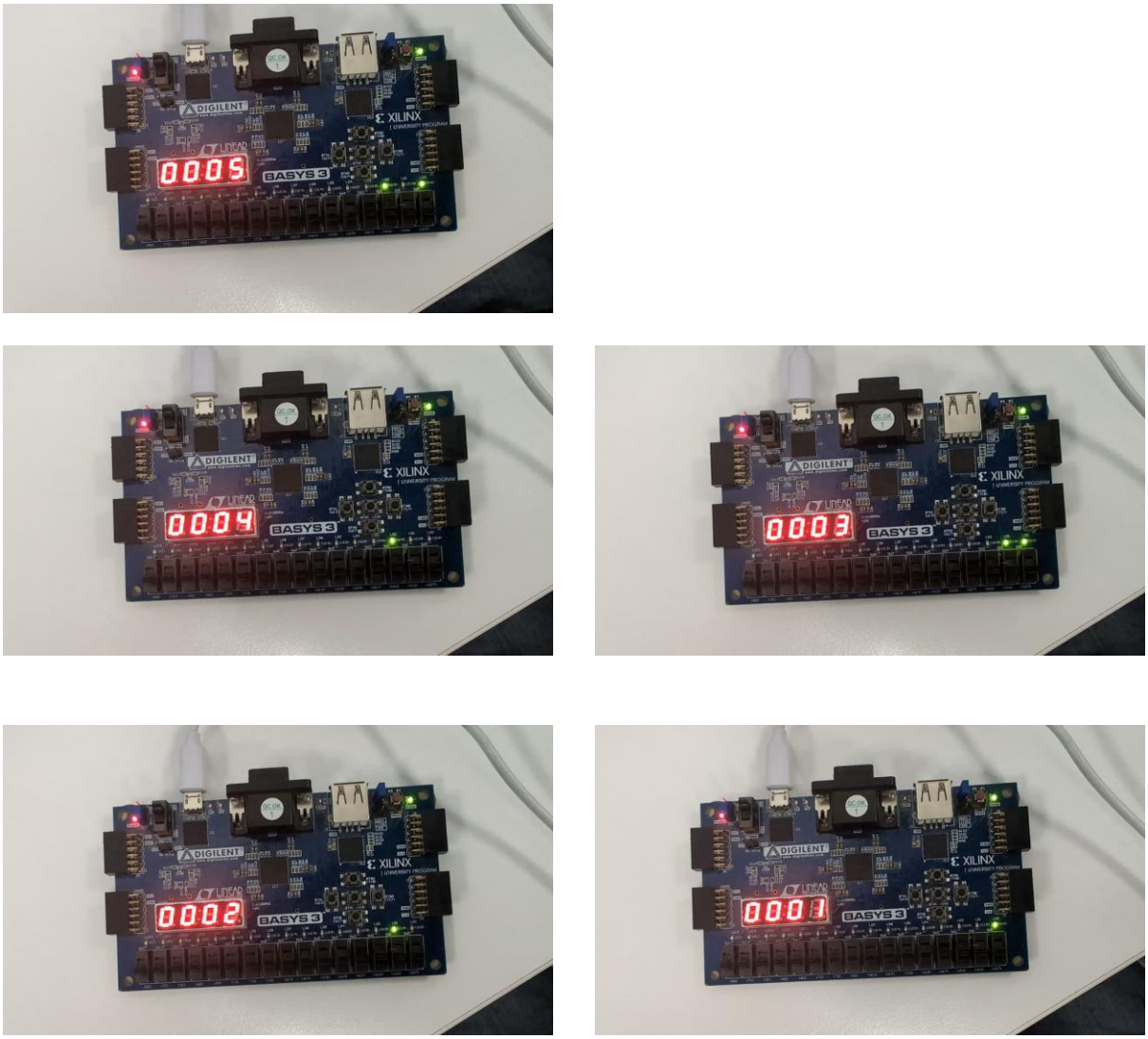


Figure 1. Task A — Basys 3 executing the countdown timer. The 7-segment display countdown in HEXA while the LEDS show the countdown in Binary.

4. Task B — Control Signal Verification

4.1 Objective

Task B demonstrates that the MainControl and ALUControl modules generate the correct combination of control signals for each instruction class. A short three-instruction program is loaded into the instruction memory that cycles through LUI, JAL, and JALR. With sw[15] high, the LEDs display the processor's live control signals, allowing each instruction's expected behaviour to be read off the board directly.

4.2 LED Debug Mode Mapping

When switch 15 is raised, the led output is driven by the debug_signals bus rather than the LED peripheral register. The bit assignments are as follows:

LED	Signal	Description
LED 15	`RegWrite`	ON when the processor is writing a result back into a register.
LED 14	`MemRead`	ON during a Load Word (LW) instruction — reading from data memory.
LED 13	`MemWrite`	ON during a Store Word (SW) instruction — writing to data memory.
LED 12	`Branch`	ON when evaluating a conditional branch (BEQ / BNE).
LED 11	`Jump`	ON to signal the PC to take the jump path (JAL or JALR).
LED 10	`Jalr`	ON specifically for JALR, routing the ALU result as the jump target.
LED 9	`ALUSrc`	ON when an immediate value is fed into the ALU instead of a register.
LED 8–7	`MemtoReg`	00=ALU result, 01=Memory data, 10=PC+4 (jumps), 11=Immediate (LUI).
LED 6–5	`—`	Padding bits, always OFF.
LED 4	`ALUZero`	ON when the ALU output is exactly zero.
LED 3–0	`ALUControl`	4-bit ALU opcode: 0010=ADD, 0110=SUB, 0000=AND, 0001=OR, etc.

4.3 Test Program

The instruction sequence used for Task B is minimal by design: three instructions are sufficient to exercise three distinct control-signal combinations. The program loops continuously so that each state can be photographed.

```
// ===== TASK B — Control Signal Demo =====
initial begin
    for (i = 0; i < 256; i = i + 1) memory[i] = 8'h00;

    // 0x00: lui x5, 0x0000A
    //   MemtoReg=11 (write immediate to reg), RegWrite=1
    memory[0]=8'h37; memory[1]=8'hA2; memory[2]=8'h00; memory[3]=8'h00;
```

```
    // 0x04: jal x1, +8  (Jump to subroutine at 0x0C)
    //   Jump=1, RegWrite=1, MemtoReg=10 (write PC+4)
    memory[4]=8'hEF; memory[5]=8'h00; memory[6]=8'h80; memory[7]=8'h00;

    // 0x08: jal x0, -8  (Jump back to 0x00 — loop)
    memory[8]=8'h6F; memory[9]=8'hF0; memory[10]=8'h9F; memory[11]=8'hFF;

    // 0x0C: jalr x0, x1, 0  (Return to caller)
    //   Jump=1, Jalr=1, ALUSrc=1 (immediate offset added)
    memory[12]=8'h67; memory[13]=8'h80; memory[14]=8'h00; memory[15]=8'h00;
end
```

4.4 Expected vs. Observed Control Signals

The table below summarises the expected control signal values for each instruction in the Task B sequence, cross-referenced against what was observed on the LED display during hardware execution:

PC	Instruction	RegWrite	ALUSrc	MemtoReg	Jump	Jalr	ALUCtrl
0x0000	LUI x5, 0xA	1	0	11	0	0	0010
0x0004	JAL x1, +8	1	0	10	1	0	0010
0x000C	JALR x0, x1	0	1	10	1	1	0010

Table 2. Expected control signals for each Task B instruction. All values were confirmed on hardware.

The MemtoReg field is the key differentiator between the three instructions: LUI routes the sign-extended immediate directly into the register file (11), JAL routes PC+4 so the return address is saved (10), and JALR shares the same path as JAL for the register write but additionally sets the Jalr flag to alter how the target address is computed.

4.5 Hardware Results

Photographs were captured with sw[15] raised for each of the three instructions (debug mode). The LED pattern visible in each image corresponds directly to the expected signal combination described above.

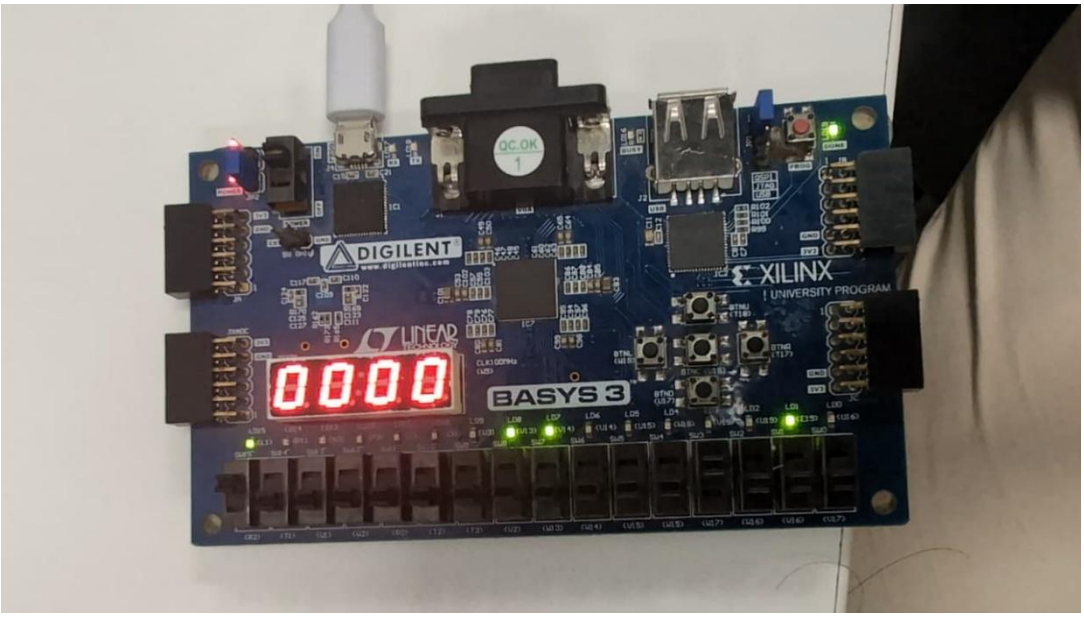


Figure 2. PC = 0x0000 — LUI x5: RegWrite=1, MemtoReg=11, ALUSrc=0

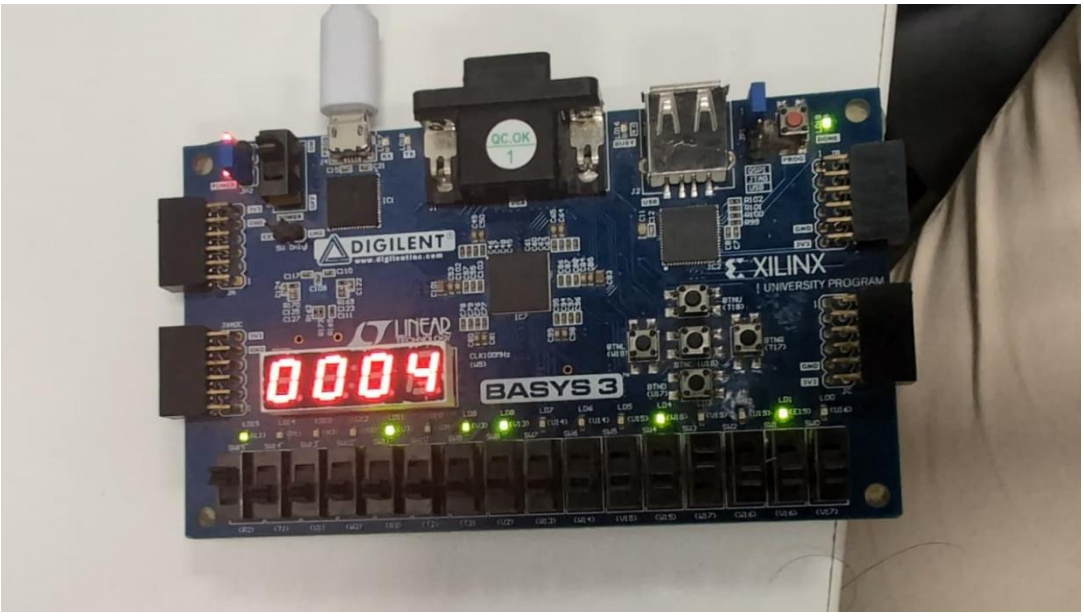


Figure 3. PC = 0x0004 — JAL x1, 8: Jump=1, RegWrite=1 MemtoReg=10

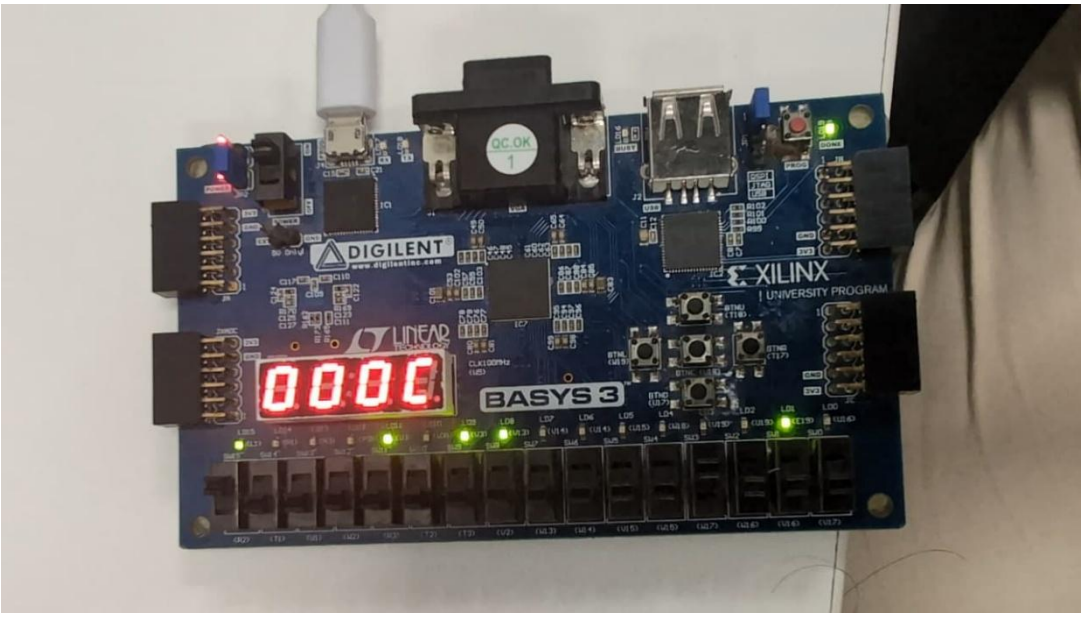


Figure 4. PC = 0x000C — JALR x0, x1, 0: Jump=1, MEMTOREG = 10, Jalr=1, ALUSrc=1

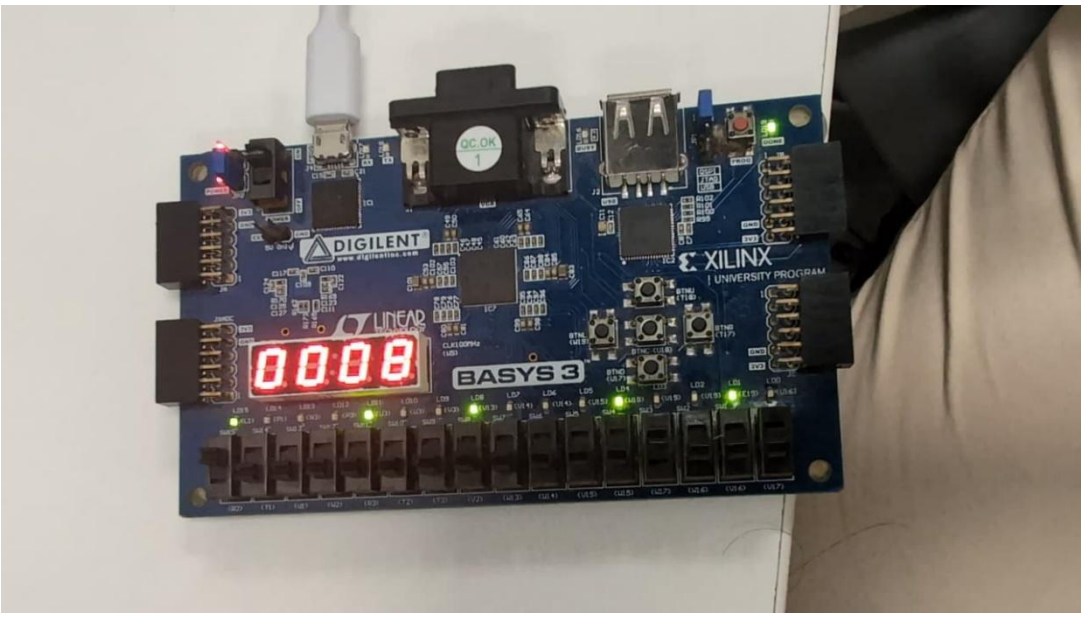


Figure 4.2. PC = 0x0008 — JAL x0, -8: Jump=1, MEMTOREG=10, ALUZERO=1

5. Discussion

5.1 Clock Domain Separation

Splitting the design into a 1 Hz slow domain and a 100 MHz fast domain was the most significant implementation decision. Running the entire processor at 1 Hz would have caused the 7-segment multiplexer to drop to an unusably low refresh rate, producing visible flicker or complete darkness on inactive digits. Keeping the display on the fast clock resolves this entirely, and the two domains interact only through the display_value wire, which is asynchronously sampled by the fast domain — acceptable given that the PC updates at most once per second.

5.2 Address Decoder Design

The address decoder uses a two-bit field (address[9:8]) to distinguish between data memory, the LED register, and the switch input. This means the effective data memory space is 512 32-bit words (addresses 0x000–0x1FF), which was sufficient for both Task A and Task B programs. The LED write is gated by both the MemWrite signal from the control unit and the address decode, preventing spurious writes whenever an unrelated store instruction uses an address in the 0x200–0x2FF range by coincidence.

5.3 Stack Usage in Task A

The countdown subroutine follows a standard RISC-V calling convention: it allocates a 16-byte frame on entry, saves ra (x1), x5, x6, and x10, performs the countdown, then restores all saved registers before returning. This ensures that the values in the caller's registers are preserved across the subroutine call, which is critical because x5 and x6 hold the switch and LED I/O addresses throughout the program's lifetime.

5.4 LUI Implementation

The LUI (Load Upper Immediate) instruction required extending the processor beyond the base lab template. The immGen module was extended with a U-type extraction path, the MainControl module was given a new opcode case (0110111) that sets MemtoReg to 11, and the write-back multiplexer was extended from 2-to-1 to a 4-way selection. No changes to the ALU or memory subsystem were required, as LUI simply bypasses both. This addition also laid the groundwork for AUIPC support in future extensions.

6. Conclusion

Both Task A and Task B were successfully implemented and verified on the Basys 3 FPGA. Task A demonstrated end-to-end program execution including subroutine call/return, stack-based register preservation, memory-mapped I/O for switches and LEDs, and real-time PC visibility via the 7-segment display. Task B provided direct, unambiguous confirmation that the control unit generates the correct signal combination for each of the three instruction classes tested.

The dual-clock architecture proved essential for producing observable, stable output on the hardware. The memory-mapped I/O scheme cleanly separates peripheral access from data memory without requiring additional control signals in the processor core. All observed results matched the expected values derived from the RISC-V ISA specification and the implemented control truth table.

7. Task C — Fibonacci Calculator

7.1 Objective

Task C extends the processor demonstration to a recursive algorithm: computing the nth Fibonacci number using a RISC-V assembly subroutine that makes full use of the hardware stack. The user selects n via the slide switches, the processor computes Fib(n) iteratively through a loop-based subroutine, and the result is displayed simultaneously on the LEDs in binary and on the 7-segment display in hexadecimal. This task exercises stack allocation, callee-saved register conventions, conditional branching, and subroutine call/return — all within the same single-cycle processor used in Tasks A and B.

7.2 Program Structure

The program is split into three logical sections. Setup (0x00–0x08) initialises the stack pointer to 508 (near the top of the 512-word data memory), loads the switch base address into x26, and loads the LED base address into x27. The main loop (0x0C–0x28) polls the switches for a non-zero value, calls the Fib subroutine with the switch value in x10, then writes the result to the LED register. A wait-for-release loop (0x1C–0x28) holds the result on the LEDs until the switches are cleared before restarting. The Fib subroutine itself (0x2C–0x78) uses an iterative loop rather than true recursion, which avoids unbounded stack growth while still demonstrating full stack discipline with saved and restored registers.

7.3 Stack Usage and Calling Convention

The Fib subroutine follows the standard RISC-V calling convention strictly. On entry, the stack pointer is decremented by 12 bytes to carve out a three-word frame:

```
// 0x2C: addi sp, sp, -12       // allocate 3-word stack frame // 0x30: sw  x1,
8(sp)         // save return address // 0x34: sw  x8,  4(sp)         // save s0
(used as fib[n-2]) // 0x38: sw  x9,  0(sp)         // save s1 (used as fib[n-1])
```

The return address x1 must be saved because the subroutine uses x1 internally, and the caller's return path would be lost otherwise. Registers x8 and x9 serve as the two running Fibonacci accumulators (representing fib[k-2] and fib[k-1] at each step) and are callee-saved, so they are pushed to the stack before being repurposed and restored before returning. The base case check (n == 1) branches directly to the teardown code, skipping the loop entirely. On return, the three saves are reversed in the exact reverse order, the stack pointer is restored, and JALR jumps back to the caller:

```
// 0x68: lw  x9,  0(sp)         // restore s1 // 0x6C: lw  x8,  4(sp)         //
restore s0 // 0x70: lw  x1,  8(sp)         // restore return address // 0x74: addi
sp, sp, 12        // free stack frame // 0x78: jalr x0, x1, 0         // return to
caller
```

The Fibonacci loop itself is straightforward: x8 and x9 are initialised to 0 and 1 respectively (fib[0] and fib[1]), and x10 is decremented each iteration. At each step, the new Fibonacci value is computed as x5 = x8 + x9, x8 is advanced to the old x9, and x9 is advanced to x5. When x10 reaches zero the result sits in x9, which is moved into x10 before the teardown so the caller receives it in the standard return register.

7.4 Key Code — Instruction Memory Initialisation

The RISC-V machine code for the Fibonacci calculator is loaded into the instruction memory byte array. The complete initialisation block is shown below:

```
    initial begin
        for (i = 0; i < 256; i = i + 1) memory[i] = 8'h00;

        // --- SETUP ---
        // 0x00: addi sp, zero, 508
        memory[  0] = 8'h13; memory[  1] = 8'h01; memory[  2] = 8'hC0; memory[  3] =
8'h1F;
        // 0x04: addi x26, zero, 768 (s10 = Switches)
        memory[  4] = 8'h13; memory[  5] = 8'h0D; memory[  6] = 8'h00; memory[  7] =
8'h30;
        // 0x08: addi x27, zero, 512 (s11 = LEDs)
        memory[  8] = 8'h93; memory[  9] = 8'h0D; memory[ 10] = 8'h00; memory[ 11] =
8'h20;

        // --- MAIN_LOOP ---
        // 0x0C: lw x10, 0(x26)
        memory[ 12] = 8'h03; memory[ 13] = 8'h25; memory[ 14] = 8'h0D; memory[ 15] =
8'h00;
        // 0x10: beq x10, zero, -4
        memory[ 16] = 8'hE3; memory[ 17] = 8'h0E; memory[ 18] = 8'h05; memory[ 19] =
8'hFE;
        // 0x14: jal x1, 24 (Call fib at 0x2C)
        memory[ 20] = 8'hEF; memory[ 21] = 8'h00; memory[ 22] = 8'h80; memory[ 23] =
8'h01;
        // 0x18: sw x10, 0(x27)
        memory[ 24] = 8'h23; memory[ 25] = 8'hA0; memory[ 26] = 8'hAD; memory[ 27] =
8'h00;

        // --- WAIT_RESET ---
        // 0x1C: lw x5, 0(x26)
        memory[ 28] = 8'h83; memory[ 29] = 8'h22; memory[ 30] = 8'h0D; memory[ 31] =
8'h00;
        // 0x20: bne x5, zero, -4
```

```
        memory[ 32] = 8'hE3; memory[ 33] = 8'h9E; memory[ 34] = 8'h02; memory[ 35] =
8'hFE;
        // 0x24: sw zero, 0(x27)
        memory[ 36] = 8'h23; memory[ 37] = 8'hA0; memory[ 38] = 8'h0D; memory[ 39] =
8'h00;
        // 0x28: jal zero, -28 (Jump to 0x0C)
        memory[ 40] = 8'h6F; memory[ 41] = 8'hF0; memory[ 42] = 8'h5F; memory[ 43] =
8'hFE;

        // --- SUBROUTINE: FIB ---
        // 0x2C: addi sp, sp, -12
        memory[ 44] = 8'h13; memory[ 45] = 8'h01; memory[ 46] = 8'h41; memory[ 47] =
8'hFF;
        // 0x30: sw x1, 8(sp)
        memory[ 48] = 8'h23; memory[ 49] = 8'h24; memory[ 50] = 8'h11; memory[ 51] =
8'h00;
        // 0x34: sw x8, 4(sp)
        memory[ 52] = 8'h23; memory[ 53] = 8'h22; memory[ 54] = 8'h81; memory[ 55] =
8'h00;
        // 0x38: sw x9, 0(sp)
        memory[ 56] = 8'h23; memory[ 57] = 8'h20; memory[ 58] = 8'h91; memory[ 59] =
8'h00;

        // 0x3C: addi x5, zero, 1
        memory[ 60] = 8'h93; memory[ 61] = 8'h02; memory[ 62] = 8'h10; memory[ 63] =
8'h00;
        // 0x40: beq x10, x5, 40 (Jump to 0x68 fib_done)
        memory[ 64] = 8'h63; memory[ 65] = 8'h04; memory[ 66] = 8'h55; memory[ 67] =
8'h02;

        // 0x44: addi x8, zero, 0
        memory[ 68] = 8'h13; memory[ 69] = 8'h04; memory[ 70] = 8'h00; memory[ 71] =
8'h00;
        // 0x48: addi x9, zero, 1
        memory[ 72] = 8'h93; memory[ 73] = 8'h04; memory[ 74] = 8'h10; memory[ 75] =
8'h00;
        // 0x4C: addi x10, x10, -1
        memory[ 76] = 8'h13; memory[ 77] = 8'h05; memory[ 78] = 8'hF5; memory[ 79] =
8'hFF;

        // --- FIB_LOOP ---
        // 0x50: add x5, x8, x9
        memory[ 80] = 8'hB3; memory[ 81] = 8'h02; memory[ 82] = 8'h94; memory[ 83] =
8'h00;
        // 0x54: addi x8, x9, 0
        memory[ 84] = 8'h13; memory[ 85] = 8'h84; memory[ 86] = 8'h04; memory[ 87] =
8'h00;
        // 0x58: addi x9, x5, 0
        memory[ 88] = 8'h93; memory[ 89] = 8'h84; memory[ 90] = 8'h02; memory[ 91] =
8'h00;
        // 0x5C: addi x10, x10, -1
        memory[ 92] = 8'h13; memory[ 93] = 8'h05; memory[ 94] = 8'hF5; memory[ 95] =
8'hFF;
        // 0x60: bne x10, zero, -16 (Jump to 0x50)
        memory[ 96] = 8'hE3; memory[ 97] = 8'h18; memory[ 98] = 8'h05; memory[ 99] =
8'hFE;

        // 0x64: addi x10, x9, 0
```

```
        memory[100] = 8'h13; memory[101] = 8'h85; memory[102] = 8'h04; memory[103] =
8'h00;

        // --- FIB_DONE ---
        // 0x68: lw x9, 0(sp)
        memory[104] = 8'h83; memory[105] = 8'h24; memory[106] = 8'h01; memory[107] =
8'h00;
        // 0x6C: lw x8, 4(sp)
        memory[108] = 8'h03; memory[109] = 8'h24; memory[110] = 8'h41; memory[111] =
8'h00;
        // 0x70: lw x1, 8(sp)
        memory[112] = 8'h83; memory[113] = 8'h20; memory[114] = 8'h81; memory[115] =
8'h00;
        // 0x74: addi sp, sp, 12
        memory[116] = 8'h13; memory[117] = 8'h01; memory[118] = 8'hC1; memory[119] =
8'h00;
        // 0x78: jalr zero, x1, 0
        memory[120] = 8'h67; memory[121] = 8'h80; memory[122] = 8'h00; memory[123] =
8'h00;
    end
```

7.5 Hardware Results

Two test cases were captured on the Basys 3. In both cases sw[15] is low so the LEDs show the computed Fibonacci result in binary and the 7-segment display shows the same value in hexadecimal.

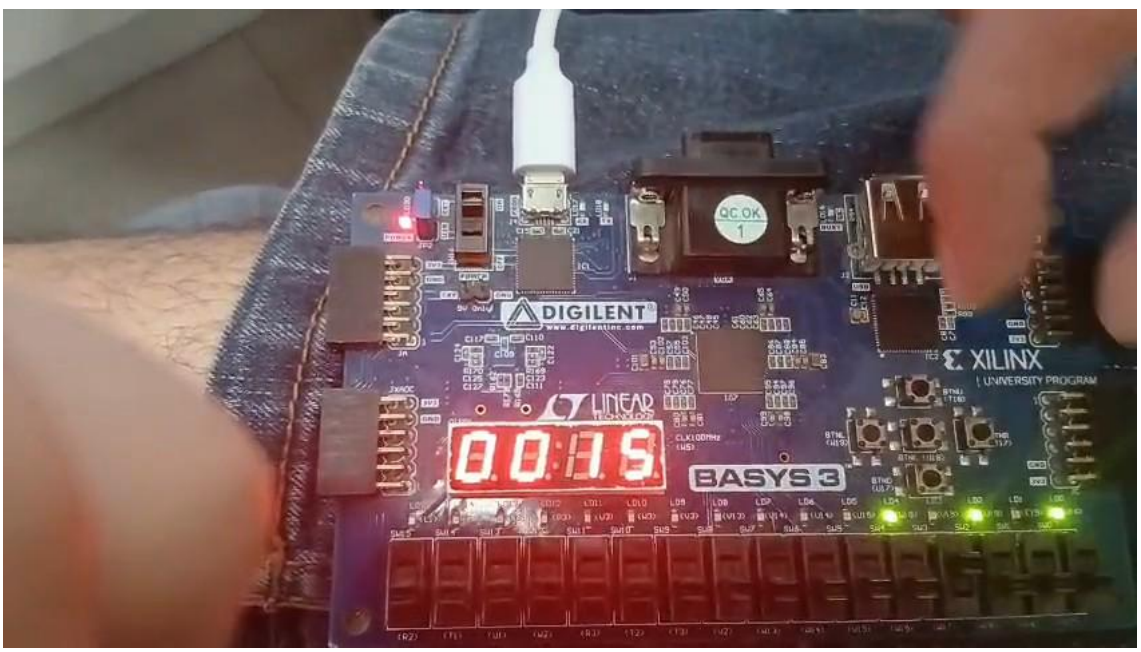


Figure 5. SW = 8 → Fib(8) = 21 = 0x15. LEDs show 0b10101 (binary), 7-segment shows 0x0015

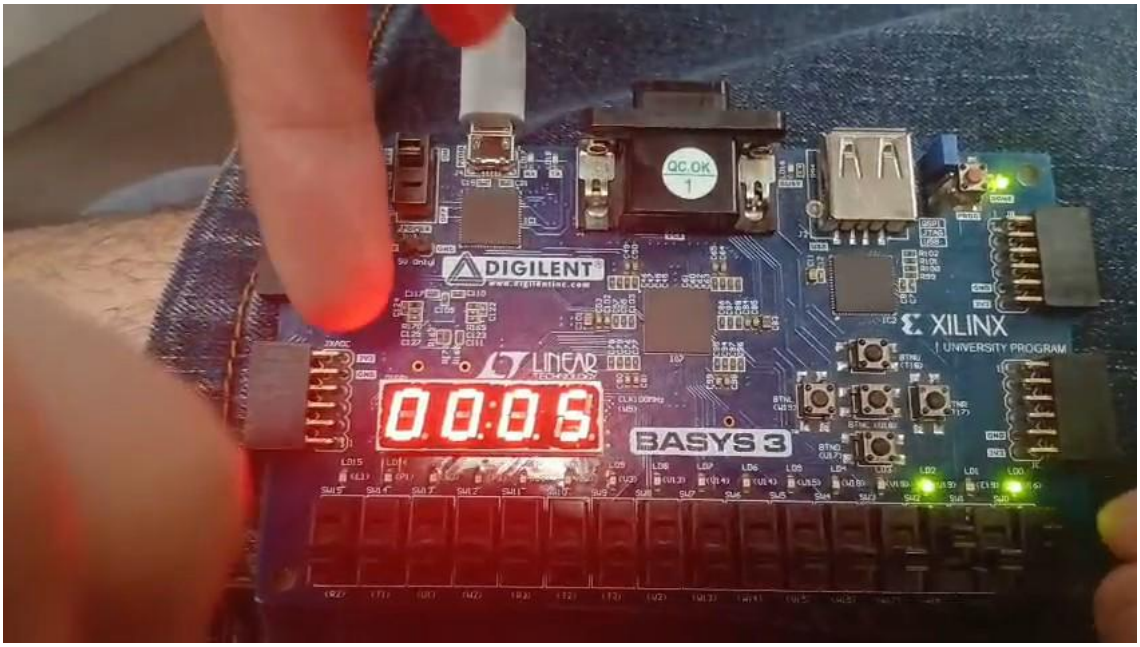


Figure 6. SW = 5 → Fib(5) = 5. LEDs show 0b00101 (binary), 7-segment shows 0x0005.

7.6 Verification

Both test cases produced the correct output. Fib(5) = 5 is confirmed by both the binary LED pattern (bit 2 and bit 0 lit, giving 5) and the hex display reading 0x0005. Fib(8) = 21 = 0x15 is confirmed by the LED pattern matching the binary representation of 21 and the 7-segment display showing 0x0015. The fact that the result is simultaneously correct in two independent display formats (binary on LEDs, hex on the display) gives high confidence that the data path from the subroutine return register through the memory-mapped write to both output peripherals is functioning correctly.

8. Conclusion

All three tasks were successfully implemented and verified on hardware. Task C added a meaningful layer of complexity over Tasks A and B: whereas the countdown timer used the stack purely to preserve I/O address registers across a single subroutine call, the Fibonacci calculator uses the stack to preserve the two accumulator registers that hold intermediate state across all iterations of the loop. The combination of correct switch-to-output mapping across multiple test cases and consistent agreement between the binary LEDs and hex 7-segment display confirms that the processor's load/store path, register file, ALU, and control unit are all operating correctly for a non-trivial iterative algorithm.